

E-Commerce ELT Pipeline

Cloud-Based Data Engineering Project

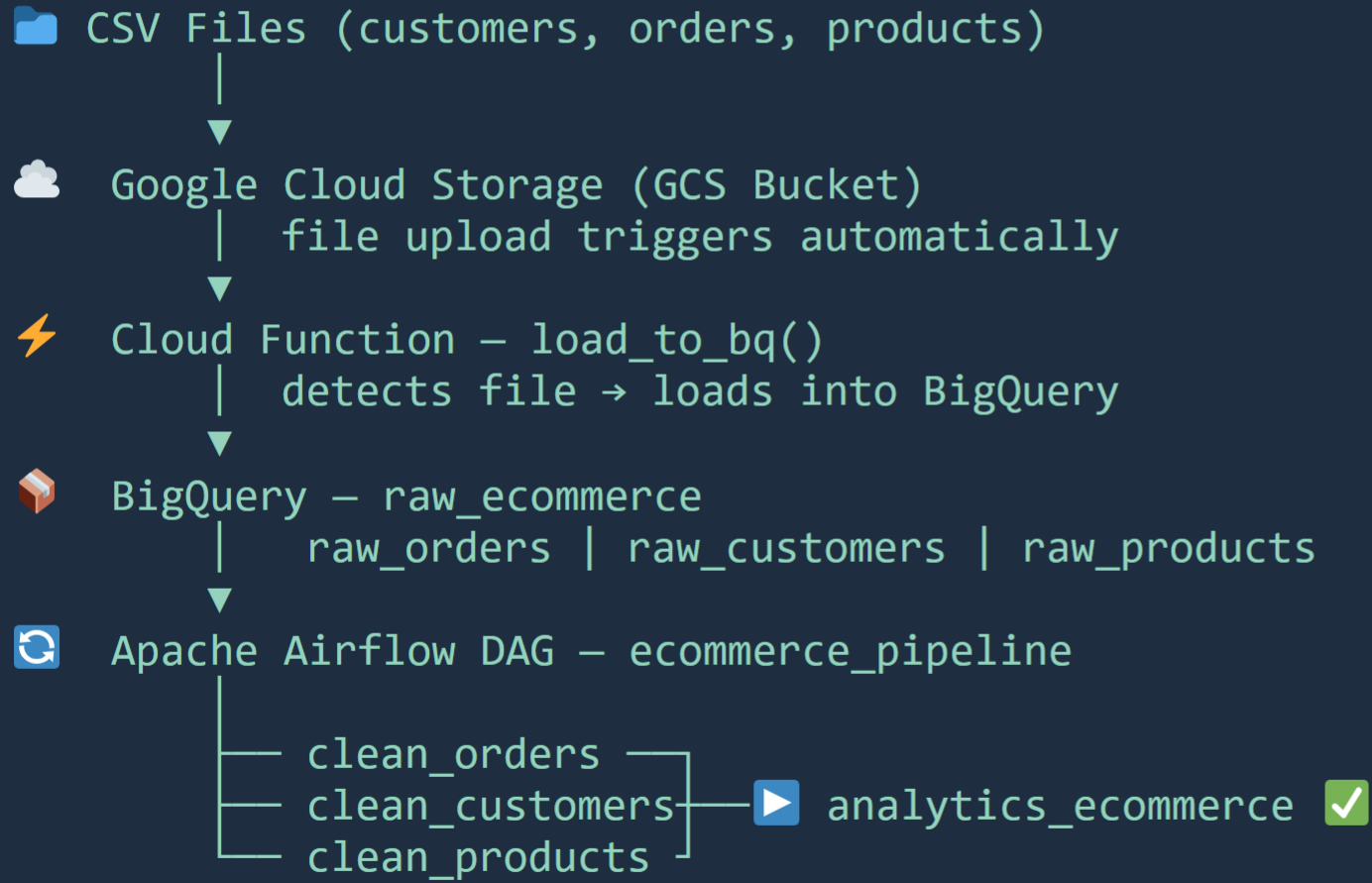
Cloud Storage · Cloud Functions · BigQuery · Apache Airflow

What I Built

A fully automated ELT pipeline that:

- Takes raw e-commerce CSV data — **customers, orders, products**
- **Auto-loads** data into BigQuery using a Cloud Function
- **Transforms and cleans** data using SQL inside BigQuery
- **Orchestrates** all tasks automatically using Apache Airflow DAG
- Produces a final **analytics-ready table** for business insights

Architecture



Dataset — Source Files

3 CSV files used to simulate real e-commerce transactions:

File	Key Fields	Rows
customers.csv	customer_id first_name last_name email city signup_date	50
orders.csv	order_id customer_id product_id quantity status order_date	500
products.csv	product_id product_name category unit_price	12

- Date range: Jan 2023 → Dec 2024
- Cities: Sydney, Houston, Berlin, London, Mumbai, New York and more
- Categories: Electronics, Furniture, Accessories, Clothing

Cloud Function — Part 1

Imports, Clients & Helper Functions

```
import functions_framework
from google.cloud import bigquery, storage
import pandas as pd
import io

PROJECT_ID = "airflow-project-1-494605"
DATASET     = "raw_ecommerce"

# Connect to GCP services – created once, reused for every request
bq_client    = bigquery.Client()    # connects to BigQuery
storage_client = storage.Client()    # connects to Cloud Storage

#  Create dataset if it doesn't exist
def create_dataset():
    dataset_id = f"{PROJECT_ID}.{DATASET}"
    dataset     = bigquery.Dataset(dataset_id)
    dataset.location = "asia-south1"
    try:
        bq_client.get_dataset(dataset_id)    # check if exists
        print("Dataset already exists")
    except:
        bq_client.create_dataset(dataset, exists_ok=True) # create if not
```

Cloud Function — Part 2

Table Routing & Main Trigger

```
# ✓ Map filename → BigQuery table name
def get_table_name(file_name):
    name = file_name.lower()
    if "orders" in name: return "raw_orders"
    elif "customers" in name: return "raw_customers"
    elif "products" in name: return "raw_products"
    else: return None # unrecognized file – skip safely

# ✓ Main Cloud Function trigger
@functions_framework.cloud_event
def load_to_bq(cloud_event):
    bucket = cloud_event.data["bucket"] # which GCS bucket triggered
    file_name = cloud_event.data["name"] # which file was uploaded

    print(f"Processing file: {file_name}")
    create_dataset() # ensure dataset exists

    table = get_table_name(file_name)
    if not table:
        print("File not recognized"); return
```

Cloud Function — Part 3

CSV / JSON Loading into BigQuery

```
# — CASE 1: CSV or JSON file —
if file_name.endswith(".csv") or file_name.endswith(".json"):
    uri = f"gs://{bucket}/{file_name}"

    job_config = bigquery.LoadJobConfig(
        autodetect      = True,
        source_format    = bigquery.SourceFormat.CSV
                        if file_name.endswith(".csv")
                        else bigquery.SourceFormat.NEWLINE_DELIMITED_JSON,
        skip_leading_rows = 1 if file_name.endswith(".csv") else 0,
        write_disposition = "WRITE_APPEND",
    )
    job = bq_client.load_table_from_uri(
        uri, table_id, job_config=job_config
    )
    job.result()
    print(f"✅ Loaded {file_name} → {table}")
```

- `autodetect=True` — BigQuery auto-detects column types from CSV

Cloud Function — Part 4

Excel Loading into BigQuery

```
# — CASE 2: Excel file (.xlsx) —
elif file_name.endswith(".xlsx"):
    # Step 1: Download file from GCS as raw bytes
    blob = storage_client.bucket(bucket).blob(file_name)
    file_bytes = blob.download_as_bytes()

    # Step 2: Read bytes into a pandas DataFrame
    df = pd.read_excel(io.BytesIO(file_bytes))

    if df.empty:
        print("Excel file is empty, skipping"); return

    # Step 3: Load DataFrame directly into BigQuery
    job = bq_client.load_table_from_dataframe(
        df,
        table_id,
        job_config = bigquery.LoadJobConfig(
            write_disposition = "WRITE_APPEND",
            autodetect         = True
        )
    )
    job.result()
    print(f"✅ Excel loaded → {table}")
```

```
also:
```

BigQuery — Raw Tables Created

After uploading all 3 CSV files to GCS the function auto-created:

Dataset	Table	Records
<code>raw_ecommerce</code>	<code>raw_orders</code>	
<code>raw_ecommerce</code>	<code>raw_customers</code>	50 rows
<code>raw_ecommerce</code>	<code>raw_products</code>	

- Data loaded **automatically** — no manual steps needed
- `autodetect=True` detected schema from CSV headers
- All 3 datasets created inside BigQuery `asia-south1` region

BigQuery — Cleaning Customers

```
CREATE OR REPLACE TABLE
`airflow-project-1-494605.cleaned_ecommerce.clean_customers` AS
SELECT
  SAFE_CAST(customer_id AS STRING)           AS customer_id,
  INITCAP(TRIM(CONCAT(first_name, ' ', last_name))) AS full_name,
  LOWER(TRIM(email))                           AS email,
  INITCAP(TRIM(city))                         AS city,
  signup_date
FROM `airflow-project-1-494605.raw_ecommerce.raw_customers`
WHERE customer_id IS NOT NULL
```

Transformations applied:

- `INITCAP(TRIM(...))` — proper case + remove whitespace on names & city
- `LOWER(TRIM(email))` — standardize email to lowercase
- `SAFE_CAST` — safe type conversion without crashing
- `WHERE customer_id IS NOT NULL` — remove invalid records

BigQuery — Cleaning Orders & Products

```
-- CLEAN ORDERS
CREATE OR REPLACE TABLE
  `airflow-project-1-494605.cleaned_ecommerce.clean_orders` AS
SELECT
  SAFE_CAST(order_id AS STRING) AS order_id,
  SAFE_CAST(customer_id AS STRING) AS customer_id,
  SAFE_CAST(product_id AS STRING) AS product_id,
  SAFE_CAST(quantity AS INT64) AS quantity,
  LOWER(TRIM(status)) AS status,
  DATE(order_date) AS order_date
FROM `airflow-project-1-494605.raw_ecommerce.raw_orders`
WHERE order_id IS NOT NULL

-- CLEAN PRODUCTS
CREATE OR REPLACE TABLE
  `airflow-project-1-494605.cleaned_ecommerce.clean_products` AS
SELECT
  SAFE_CAST(product_id AS STRING) AS product_id,
  INITCAP(TRIM(product_name)) AS product_name,
  INITCAP(TRIM(category)) AS category,
  SAFE_CAST(unit_price AS FLOAT64) AS unit_price
FROM `airflow-project-1-494605.raw_ecommerce.raw_products`
WHERE product_id IS NOT NULL
```

BigQuery — Final Analytics Table

```
CREATE OR REPLACE TABLE
`airflow-project-1-494605.final_ecommerce.analytics_ecommerce` AS
SELECT DISTINCT
  o.order_id, o.order_date,
  EXTRACT(YEAR FROM o.order_date) AS order_year,
  EXTRACT(MONTH FROM o.order_date) AS order_month,
  o.status, o.quantity,
  c.full_name, c.email, c.city,
  p.product_name, p.category, p.unit_price,
  (o.quantity * p.unit_price) AS total_amount,
  CASE
    WHEN (o.quantity * p.unit_price) > 1000 THEN 'High Purchase'
    WHEN (o.quantity * p.unit_price) > 500 THEN 'Medium Purchase'
    ELSE 'Low Purchase'
  END AS order_value_category
FROM `cleaned_ecommerce.clean_orders` o
LEFT JOIN `cleaned_ecommerce.clean_customers` c ON o.customer_id = c.customer_id
LEFT JOIN `cleaned_ecommerce.clean_products` p ON o.product_id = p.product_id
```

- **SELECT DISTINCT** — removes duplicate records from repeated uploads
- **LEFT JOIN** — keeps all orders even if customer/product data is missing

Airflow DAG — Orchestration

```
from airflow import DAG
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator
from datetime import datetime

with DAG(
    dag_id          = "ecommerce_pipeline",
    start_date      = datetime(2024, 1, 1),
    schedule_interval = None,          # triggered manually
    catchup         = False,          # no backfill on old dates
) as dag:

    # 3 cleaning tasks run in PARALLEL
    clean_orders      = BigQueryInsertJobOperator(task_id="clean_orders", ...)
    clean_customers   = BigQueryInsertJobOperator(task_id="clean_customers", ...)
    clean_products    = BigQueryInsertJobOperator(task_id="clean_products", ...)
    final_table       = BigQueryInsertJobOperator(task_id="final_table", ...)

    # Dependency – all 3 must finish before final_table runs
    [clean_orders, clean_customers, clean_products] >> final_table
```

Airflow DAG — Task Flow



Each task uses `BigQueryInsertJobOperator` with:

Config	Value
<code>useLegacySql</code>	<code>False</code>
<code>location</code>	<code>asia-south1</code> — matches dataset region
<code>project_id</code>	<code>airflow-project-1-494605</code>
<code>schedule_interval</code>	<code>None</code> — manual trigger

Triggered from Airflow UI at `localhost:8080`

— Key Insights

Metric	Value
 Total Revenue	₹2,35,995
 Total Orders	500
 Total Units Sold	1,544
 Top Product	Laptop — ₹1,07,990
 Top City	Sydney — 16.3%

Key observations:

-  Peak sales in **Jul 2023** (~₹24,000) and **Oct 2024** (~₹20,000)
-  **Laptop dominates** — contributes 45% of total revenue
-  Revenue spread evenly across **10 global cities**
-  Consistent monthly revenue between **₹8,000 – ₹15,000**

Datasets in BigQuery — Final Structure

```
airflow-project-1-494605/
├── raw_ecommerce/
│   ├── raw_orders      ← loaded by Cloud Function (500 rows)
│   ├── raw_customers   (50 rows)
│   └── raw_products    (12 rows)
├── cleaned_ecommerce/  ← transformed by SQL in BigQuery
│   ├── clean_orders
│   ├── clean_customers
│   └── clean_products
└── final_ecommerce/    ← orchestrated by Airflow DAG
    └── analytics_ecommerce ✓ (500 rows, joined & clean)
```

Challenges I Faced

- `google_cloud_default` `conn_id not found` — fixed by adding GCP connection in Airflow Admin → Connections
- `Duplicate records` — caused by `WRITE_APPEND` on re-uploads, fixed using `SELECT DISTINCT` in final table
- `Location mismatch` — BigQuery dataset was in `US` but data in `asia-south1`, fixed in Query Settings
- `DAG syntax errors` — missing `AS` keyword, wrong quotes, typos in operator names fixed one by one
- `409 conflict error` — dataset already existed, fixed by adding `exists_ok=True` in `create_dataset()`
- `elif indentation error` — `elif` was inside `if` block instead of same level

Key Learnings

- **Event-driven pipelines** run automatically — no manual steps needed
- **ELT pattern** — load raw data first, transform inside BigQuery using SQL
- `SAFE_CAST` prevents pipeline crashes from bad or mismatched data types
- `exists_ok=True` handles dataset creation safely without errors
- **Airflow DAGs** enable parallel task execution and dependency management
- `SELECT DISTINCT` is essential when using `WRITE_APPEND` to avoid duplicates
- **IAM permissions** control exactly what each GCP service can access
- **Location must match** — all GCP resources must be in same region

Conclusion

What I built is a **fully automated end-to-end ELT pipeline on GCP**:

- Raw CSV data uploaded to **Cloud Storage**
- **Cloud Function** auto-triggers and loads data into BigQuery
- **SQL transformations** clean and standardize the data in 3 datasets
- **Apache Airflow DAG** orchestrates all tasks automatically
- Final **analytics table** with ₹2,35,995 revenue insights ready for reporting

| *"Raw data becomes business intelligence — automatically."*

Thank You

E-Commerce ELT Pipeline on Google Cloud Platform

Built using:

-  Google Cloud Storage
-  Cloud Functions (Python)
-  Google BigQuery
-  Apache Airflow

Thank you for watching!